

# LAB 05

## NHIỄU, LỌC ẢNH VÀ KHÔI PHỤC ẢNH

Giảng viên biên soạn: Võ Quang Hoàng Khang

### 1. Mục tiêu bài thực hành

- Giải thích nguyên lý lọc ảnh trong miền không gian và vai trò của kernel.
- Phân biệt correlation và convolution.
- Mô phỏng Gaussian Noise và Salt & Pepper Noise với nhiều mức nhiễu.
- Thực hiện Mean/Box Filter, Gaussian Filter và Median Filter.
- Phân tích ảnh hưởng của kernel size và sigma đến khả năng khử nhiễu và bảo toàn chi tiết.
- Lựa chọn bộ lọc phù hợp với từng loại nhiễu dựa trên cơ chế và thực nghiệm.
- Thực hiện sharpening bằng Laplacian và Unsharp Masking.
- Tự cài và sử dụng MSE, PSNR; sử dụng SSIM để đánh giá ảnh phục hồi.
- Xây dựng benchmark so sánh nhiều bộ lọc và pipeline khôi phục ảnh cũ.

### 2. Chuẩn bị và môi trường

Sinh viên sử dụng Google Colab. Chuẩn bị ít nhất 02 ảnh sạch làm ảnh tham chiếu và 01 ảnh cũ/ảnh chất lượng thấp để thực hiện bài tích hợp.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.metrics import structural_similarity

print("OpenCV:", cv2.__version__)
print("NumPy :", np.__version__)
```

### 3. Phần A - Filtering trong miền không gian

#### Bài 1. Kernel và phép lọc

Xét Mean Kernel  $3 \times 3$ :  $K = (1/9)[[1,1,1],[1,1,1],[1,1,1]]$ . Mỗi pixel đầu ra được tính từ lân cận của pixel tương ứng ở ảnh đầu vào.

```
gray = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

kernel = np.ones((3, 3), dtype=np.float32) / 9.0

result = cv2.filter2D(
    gray,
    ddepth=-1,
    kernel=kernel
)
```

- Giải thích vì sao tổng hệ số của Mean Kernel bằng 1.
- Giải thích vì sao lọc trung bình làm ảnh mờ.
- So sánh kernel  $3 \times 3$ ,  $5 \times 5$ ,  $9 \times 9$ .
- Nhận xét mối quan hệ giữa smoothing và mất chi tiết.

## Bài 2. Tính thủ công một pixel

Cho vùng ảnh  $I = \begin{bmatrix} 10 & 20 & 30 \\ 20 & 30 & 40 \\ 30 & 40 & 50 \end{bmatrix}$  và Mean Kernel  $3 \times 3$ . Tính thủ công giá trị pixel trung tâm bằng tổng tích từng phần tử, sau đó kiểm tra bằng Python.

```
patch = np.array([
    [10,20,30],
    [20,30,40],
    [30,40,50]
], dtype=np.float32)

kernel = np.ones((3,3), np.float32) / 9.0

value = np.sum(patch * kernel)
print("Filtered center value:", value)
```

## Bài 3. Correlation và Convolution

Correlation trượt kernel trực tiếp trên ảnh; convolution lật kernel theo hai chiều trước khi trượt. Với kernel đối xứng như Gaussian/Mean, hai phép có thể cho kết quả giống nhau.

- Giải thích vì sao với kernel đối xứng, việc lật kernel không làm thay đổi kernel.
- Cho một kernel không đối xứng và dự đoán correlation/convolution có khác nhau không.

# 4. Phần B - Mô phỏng nhiễu

## Bài 4. Gaussian Noise

Mô hình:  $g(x,y)=f(x,y)+n(x,y)$ , với  $n$  tuân theo phân phối chuẩn  $N(\text{mean}, \text{sigma}^2)$ .

```
def add_gaussian_noise(image, mean=0, sigma=20, seed=None):
    rng = np.random.default_rng(seed)

    noise = rng.normal(
        mean,
        sigma,
        image.shape
    )

    noisy = image.astype(np.float32) + noise

    return np.clip(
        noisy,
        0,
        255
    ).astype(np.uint8)

noisy_g = add_gaussian_noise(gray, sigma=20, seed=42)
```

- Thử  $\sigma = 5, 10, 20, 40$ .
- Dự đoán trước khi chạy:  $\sigma$  tăng thì mức biến thiên nhiều thay đổi thế nào?
- Giữ seed cố định khi cần so sánh công bằng các bộ lọc.

## Bài 5. Salt & Pepper Noise

```
def add_salt_pepper_noise(image, amount=0.05, seed=None):
    rng = np.random.default_rng(seed)
    noisy = image.copy()

    random_matrix = rng.random(image.shape[:2])

    noisy[random_matrix < amount / 2] = 0
    noisy[random_matrix > 1 - amount / 2] = 255

    return noisy

noisy_sp = add_salt_pepper_noise(gray, amount=0.05, seed=42)
```

- Thử  $\text{amount} = 0.01, 0.03, 0.05, 0.10$ .
- Salt tương ứng pixel trắng 255; Pepper tương ứng pixel đen 0.
- So sánh trực quan đặc điểm Gaussian Noise và impulse noise.

## 5. Phần C - Mean và Gaussian Filter

### Bài 6. Mean/Box Filter

```
mean3 = cv2.blur(noisy_g, (3,3))
mean5 = cv2.blur(noisy_g, (5,5))
mean9 = cv2.blur(noisy_g, (9,9))
```

- Hiện thị Noisy |  $3 \times 3$  |  $5 \times 5$  |  $9 \times 9$ .
- Nhận xét Noise Reduction và Detail Loss.
- Giải thích vì sao kernel lớn hơn thường làm mượt mạnh hơn nhưng mất cạnh nhiều hơn.

### Bài 7. Gaussian Filter

Gaussian Filter gán trọng số lớn hơn cho pixel gần tâm. Hàm Gaussian 2D:  $G(x,y) = 1/(2\pi\sigma^2) \cdot \exp(-(x^2+y^2)/(2\sigma^2))$ .

```
gaussian = cv2.GaussianBlur(
    noisy_g,
    (5,5),
    sigmaX=1.0
)
```

- So sánh với Mean Filter cùng kernel size.
- Giải thích vai trò của  $\sigma$ .

### Bài 8. Khảo sát sigma

```
sigma_values = [0.5, 1.0, 2.0, 4.0]

results = {}
```

```

for sigma in sigma_values:
    results[sigma] = cv2.GaussianBlur(
        noisy_g,
        (9,9),
        sigmaX=sigma
    )

```

| Sigma | Noise | Blur | Detail | Nhận xét |
|-------|-------|------|--------|----------|
| 0.5   |       |      |        |          |
| 1.0   |       |      |        |          |
| 2.0   |       |      |        |          |
| 4.0   |       |      |        |          |

## 6. Phần D - Median Filter và so sánh

### Bài 9. Median Filter

```

median3 = cv2.medianBlur(noisy_sp, 3)
median5 = cv2.medianBlur(noisy_sp, 5)
median7 = cv2.medianBlur(noisy_sp, 7)

```

Median Filter thay giá trị pixel trung tâm bằng trung vị của các giá trị trong cửa sổ, thay vì trung bình. Vì vậy các giá trị xung bất thường 0/255 thường ít chi phối kết quả hơn.

- Giải thích vì sao Median Filter phù hợp với Salt & Pepper Noise.
- Giải thích vì sao kích thước kernel thường là số lẻ.

### Bài 10. Mean vs Gaussian vs Median

```

mean_f = cv2.blur(noisy_sp, (3,3))
gauss_f = cv2.GaussianBlur(noisy_sp, (3,3), 1.0)
median_f = cv2.medianBlur(noisy_sp, 3)

```

- Thực hiện trên cả Gaussian Noise và Salt & Pepper Noise.
- Không chỉ chọn bằng mắt; cần giải thích dựa trên cơ chế filter và metrics ở phần sau.

## 7. Phần E - Sharpening

### Bài 11. Laplacian Sharpening

```

laplacian = cv2.Laplacian(
    gray,
    cv2.CV_64F
)

sharp = gray.astype(np.float64) - laplacian

sharp = np.clip(
    sharp,
    0,
    255
).astype(np.uint8)

```

- Hiện thị Original | Laplacian | Sharpened.
- Giải thích vì sao đạo hàm bậc hai phản ứng mạnh tại vùng intensity thay đổi nhanh.
- Quan sát khả năng khuếch đại noise.

## Bài 12. Unsharp Masking

Detail = Original - Blur; Sharpened = Original + amount × Detail.

```
def unsharp_mask(image, sigma=1.0, amount=1.0):
    blurred = cv2.GaussianBlur(
        image,
        (0,0),
        sigma
    )

    sharp = cv2.addWeighted(
        image,
        1 + amount,
        blurred,
        -amount,
        0
    )
    return sharp

for amount in [0.5, 1.0, 2.0]:
    out = unsharp_mask(gray, sigma=1.0, amount=amount)
    print("amount =", amount)
```

- Nhận xét over-sharpening/halo khi amount quá lớn.
- Thử sharpening ảnh noisy và ảnh đã denoise để so sánh.

## 8. Phần F - Đánh giá MSE, PSNR và SSIM

### Bài 13. Mean Squared Error - MSE

$MSE = (1/MN) \sum [I(x,y) - K(x,y)]^2$ . MSE càng nhỏ thì sai số bình phương trung bình so với ảnh tham chiếu càng nhỏ.

```
def mse(img1, img2):
    diff = (
        img1.astype(np.float64)
        - img2.astype(np.float64)
    )
    return np.mean(diff ** 2)
```

### Bài 14. Peak Signal-to-Noise Ratio - PSNR

Với ảnh 8-bit:  $PSNR = 10 \log_{10}(255^2/MSE)$ . PSNR tăng thường tương ứng sai số MSE giảm, nhưng không bảo đảm chất lượng cảm nhận luôn tốt hơn.

```
def my_psnr(original, restored):
    error = mse(original, restored)

    if error == 0:
```

```

        return float("inf")

    return 10 * np.log10(
        (255.0 ** 2) / error
    )

print("My PSNR:", my_psnr(gray, mean3))
print("OpenCV PSNR:", cv2.PSNR(gray, mean3))

```

## Bài 15. Structural Similarity Index - SSIM

SSIM xem xét các thành phần liên quan đến luminance, contrast và structure. Giá trị càng gần 1 thường biểu thị mức tương đồng cấu trúc cao hơn so với ảnh tham chiếu.

```

score = structural_similarity(
    gray,
    mean3,
    data_range=255
)

print("SSIM:", score)

```

- PSNR/MSE và SSIM đều cần ảnh reference khi sử dụng theo cách trong Lab này.
- Không đánh đồng một metric đơn lẻ với chất lượng cảm nhận.

## 9. Phần G - Benchmark so sánh filter

### Bài 16. Hàm đánh giá filter

```

def evaluate_filter(original, restored):
    return {
        "MSE": mse(original, restored),
        "PSNR": my_psnr(original, restored),
        "SSIM": structural_similarity(
            original,
            restored,
            data_range=255
        )
    }

```

### Bài 17. Thực nghiệm Gaussian Noise

Với sigma = 5, 10, 20, 40: tạo ảnh noisy, áp dụng Mean, Gaussian và Median, sau đó tính MSE/PSNR/SSIM.

### Bài 18. Thực nghiệm Salt & Pepper Noise

Với amount = 0.01, 0.03, 0.05, 0.10: lặp lại quy trình và so sánh.

| Noise    | Filter   | Kernel | PSNR | SSIM | Nhận xét |
|----------|----------|--------|------|------|----------|
| Gaussian | Mean     | 3×3    |      |      |          |
| Gaussian | Gaussian | 3×3    |      |      |          |
| Gaussian | Median   | 3      |      |      |          |
| S&P      | Mean     | 3×3    |      |      |          |
| S&P      | Gaussian | 3×3    |      |      |          |

|     |        |   |  |  |  |
|-----|--------|---|--|--|--|
| S&P | Median | 3 |  |  |  |
|-----|--------|---|--|--|--|

- Filter nào tốt hơn cho Gaussian Noise trong bộ dữ liệu thử nghiệm?
- Filter nào tốt hơn cho Salt & Pepper Noise?
- Khi mức nhiễu tăng, kernel/parameter tối ưu có thay đổi không?

## 10. Bài tích hợp - Image Restoration Benchmark

```
def restoration_benchmark(original, noisy):
    candidates = {
        "Mean 3x3": cv2.blur(noisy, (3,3)),
        "Gaussian 3x3": cv2.GaussianBlur(noisy, (3,3), 1.0),
        "Median 3": cv2.medianBlur(noisy, 3)
    }

    scores = {}

    for name, restored in candidates.items():
        scores[name] = evaluate_filter(
            original,
            restored
        )

    return candidates, scores
```

- Tự động tạo bảng kết quả.
- Chọn phương pháp tốt nhất theo từng metric và thảo luận nếu các metric không cùng chọn một phương pháp.
- Hiện thị Original | Noisy | các ảnh Restored.

## 11. Ứng dụng liên ngành/thực tế - Khôi phục ảnh cũ

Cho một ảnh cũ có nhiễu, tương phản thấp và hơi mờ. Kết hợp kiến thức Lab 02 và Lab 03 để đề xuất pipeline. Không bắt buộc một thứ tự duy nhất; sinh viên phải thử nghiệm và biện luận.

Pipeline gợi ý: Old Image → Denoise → CLAHE → Gamma → Sharpen → Restored.

- Thử đổi thứ tự Sharpen và Denoise để quan sát failure case.
- Giải thích vì sao sharpening trước denoise có thể khuếch đại nhiễu.
- Nếu không có ảnh gốc sạch của ảnh cũ, không được báo cáo PSNR/SSIM như thể có ground truth; thay vào đó cần đánh giá định tính hoặc dùng bộ dữ liệu mô phỏng có reference để kiểm chứng phương pháp.

## 12. Bài nâng cao

**Challenge 1 - Tự cài Mean Filter:** Không dùng cv2.blur() hoặc cv2.filter2D(); xử lý cửa sổ  $k \times k$  và so sánh với OpenCV.

**Challenge 2 - Tự cài Median Filter:** Tự lấy lân cận, sắp xếp/chọn median; đối chiếu với cv2.medianBlur().

**Challenge 3 - Parameter Search:** Tự động tìm Filter  $\times$  Kernel  $\times$  Parameter có PSNR/SSIM tốt nhất trên nhiều mức nhiễu.

**Challenge 4 - Noise Classification:** Thiết kế heuristic đơn giản để phân biệt Gaussian Noise và Salt & Pepper Noise rồi chọn filter tương ứng.

**Challenge 5 - Batch Processing:** Đọc nhiều ảnh từ thư mục, tạo nhiễu/khôi phục, tính metrics và xuất bảng CSV để chuẩn bị cho Project khôi phục ảnh.

## 13. Câu hỏi phân tích bắt buộc

1. Kernel là gì và vai trò của kernel trong spatial filtering?
2. Kernel size ảnh hưởng khả năng giảm nhiễu và bảo toàn chi tiết như thế nào?
3. Correlation và convolution khác nhau ở thao tác nào?
4. Gaussian Noise được đặc trưng bởi mean và sigma như thế nào?
5. Salt & Pepper Noise khác Gaussian Noise về hình thái ra sao?
6. Vì sao Mean Filter thường làm mờ cạnh?
7. Gaussian Filter khác Mean Filter ở cách gán trọng số như thế nào?
8. Sigma ảnh hưởng Gaussian Filter ra sao?
9. Vì sao Median Filter đặc biệt phù hợp với Salt & Pepper Noise?
10. Vì sao kernel Median thường cần kích thước lẻ?
11. Sharpening có thể làm nhiễu tăng không? Vì sao?
12. MSE phản ánh điều gì?
13. PSNR cao có luôn đồng nghĩa chất lượng cảm nhận tốt hơn không?
14. SSIM khác PSNR về ý tưởng đánh giá như thế nào?
15. Tại sao cần ảnh gốc/reference để tính PSNR và SSIM theo cách thực hành trong Lab?
16. Tại sao nhiều pipeline nên denoise trước sharpening?

## 14. Tài liệu đọc thêm

[1] OpenCV - Smoothing Images / Image Filtering:

[https://docs.opencv.org/4.x/d4/d13/tutorial\\_py\\_filtering.html](https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html)

[2] OpenCV - Image Gradients: [https://docs.opencv.org/4.x/d5/d0f/tutorial\\_py\\_gradients.html](https://docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html)

[3] scikit-image - Image Quality Metrics: <https://scikit-image.org/docs/stable/api/skimimage.metrics.html>

[4] NumPy Documentation: <https://numpy.org/doc/stable/>

[5] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., Pearson.